An evaluation of the ongoing utility of memory allocator in single- and multi-threaded contexts

Margaret Bauman
Boston University
ECE department
Boston,MA
msbauman@bu.edu

Niantong Dong
Boston University
ECE department
Boston,MA
niantongdong@gmail.com

***Abstract*—In this paper, we evaluate the premise that each memory allocator has unique advantages, especially in multi-threaded, high concurrency cases. After a series of experiments, we found that jemalloc, released in 2006, still shows relatively good performance in most benchmarks after more than a decade of development. Other comparative allocators released after jemalloc and also updated since their release, such as Hoard, do not perform as well as their papers suggest. We also found that the new memory allocator, mimalloc, introduced by Microsoft in July 2019, performed well in all experiments. But there are few relevant papers in the literature, and we have no way to fully determine its stability and scalability at present. So we think that jemalloc is still a good choice in most usage scenarios, and, although its performance in some benchmarks is not as good as mimalloc, the difference is not particularly large.**

Keywords—Memory allocation, memory management

I. INTRODUCTION

A good memory allocator can be more effective in saving time, improving program efficiency and saving memory usage when it fits the application and workload. Different memory allocator designs also provide different performance when it comes to finding free blocks and tracking used memory. Especially with the growth of data and cloud computing centers, large programs and heavy workloads require appropriate memory allocators to achieve better performance. So far, several large enterprises, such as Microsoft (mimalloc) and Facebook (jemalloc) have developed their own memory allocators to meet their specific needs. Since these are open source projects, we can also use the same allocators as the large companies for our own projects. Indeed, the 'market' provides a wide range of allocators from which to choose. Therefore, finding the most suitable allocator for a particular scenario becomes an important task. In this article, we show that, while the mainstream allocators we examine have individual strengths and weaknesses, they all perform sufficiently well in multi-threaded contexts, albeit not equally well in all situations or on all parameters.

We have chosen the most popular allocators on the market today, from the one that comes with the system to the latest allocator developed by Microsoft, mimalloc. Each has a different design and allocation policy. At the same time, except for the system allocator, all of them claim that they will be very effective in optimizing and extending multi-threaded tasks. However, there are few papers on this topic, especially those that study all four simultaneously. Most of the papers we found compared one of them with older and more outdated allocators. After more than a decade of development and constant updating, those papers may not be as convincing as they once were. Therefore, the allocators we chose represent the latest versions of known quantities (the system allocator, jemalloc, tcmalloc, and Hoard), as well as the relatively new mimalloc. We also use benchmarks that test the allocators in various ways. From simulations to server requests to ultra-high volume work, we believe the benchmark we chose should cover most of the daily use scenarios.

We will start with a brief introduction of how each of our chosen allocators works. Each allocator uses a different allocation method and design structure. We will not make any evaluation of these because each design has its advantages and disadvantages. More often than not, it is a compromise that the designer makes when faced with a certain scenario. But we will make a preliminary inference based on these designs and workings to assume which allocator will have a better benefit ratio in a given benchmark. After that we will introduce the computer configuration and system settings we used in this project and some necessary libraries. We will also briefly introduce the benchmarks we are using and why we are using them.

After that, we present our hypothesis. Then we show our conclusions from our experiments, which we analyze to understand each allocator's behavior. Our results were not identical to those in the papers we read, even the mimalloc paper, which we extend by reporting resident set size (Rss) as a measure of total memory usage and page reclaims as a measure of memory fullness.

II. MEMORY ALLOCATOR INTRODUCTION

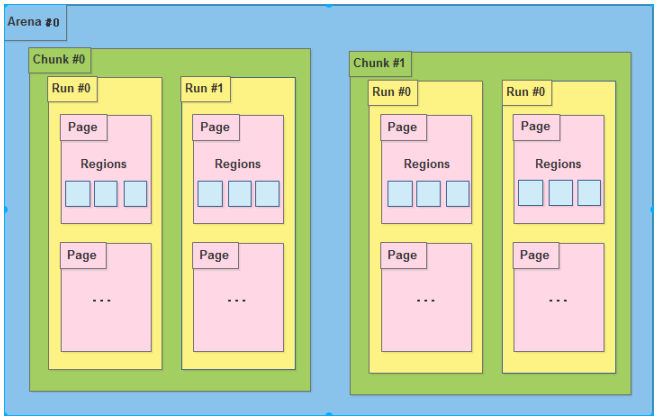
1. Malloc, system built-in

The implementation of the malloc function that comes with the Linux system has a function, also called malloc, that joins available memory blocks into a long list of so-called free lists. When the malloc function is called, it looks for a block of memory along the list that is large enough to satisfy the user's request size. Then, the block is divided in two, one block is equal in size to the size of the user request, and the other block is the size of the remaining bytes. Then, the chunk of memory allocated to the user is passed to the user program and the remaining chunk (if any) will return to the free list. When the free function is called, it joins the block of memory freed by the user to the free list. Eventually, the free list will contain many small memory pieces, and if the user requests a large memory piece at this point, there may not be any pieces in the free list that can satisfy the user's request. So, the malloc function will delay the request and start going through the memory segments in the free list, sorting them and merging the adjacent small free blocks into larger ones. The malloc function returns a NULL pointer if it cannot get a memory

block that meets the requirements, so when calling malloc to request a memory block dynamically, one must be sure to include a return value validation to see if the malloc actually returns a valid block address. Otherwise, it can cause memory issues, throw errors when accessing invalid addresses, and kill the program process.[5][6]

2. Jemalloc

Jemalloc is a modern memory allocator with the ability to allocate memory efficiently in multiple threads. In a multicore system, the most significant barrier in process efficiency has become figuring out how to effectively use several threads while avoiding locks. In a multithreaded context, jemalloc uses lock avoidance as much as feasible to speed up memory accesses and releases. The figure below is



from [1].

Jemalloc has a hierarchical memory management architecture, as shown in the diagram, with arena allocation area, Chunk, Run and page, with varied size memory blocks corresponding to different allocation regions. Each thread has its own chunk, which is in charge of quickly allocating and freeing memory blocks utilized by the current thread in order to avoid lock conflict and thread synchronization. The arena uses the idea of memory pool to partition and manage the memory area in a way that will allow for efficient management of memory blocks of various sizes with little memory fragmentation. Here are some features about the overall design of jemalloc arena, chunks, runs and pages[1]:

- A certain number of arenas are in charge of memory. Arena is typically Number of CPU * 4 and is used in multicore systems to minimize CPU cache access problems. That is, on a 4-core CPU, if a process has 16 or fewer threads, one arena is assigned to each thread to avoid access conflict.
- A large arena is divided into multiple chunks, each of which is usually 2 or 4 M
- A chunk is made up of a number of runs, which are the basic units for allocating memory in small chunks.
- A run is made up of pages that are eventually separated into a set of regions. This area is the the actual memory given to the user for small memory requests

For object allocation, jemalloc implements different kinds of memory allocation methods.

- Small object: small object allocation will start with regions first. If it is unable to find a block on regions, it will return to Regions and request new regions for that. In general, it will not request a further region unless the current run is full and create a new run for that.
- Large object: The large object allocation will check if it exceeds the max size of Run, if not, it will just request a couple of pages to store it. Otherwise, it will request from chunk for more runs for that.
- Huge object: The huge object will not touch the arena entirely, instead it will request from system memory using mmap and use red-black tree to trace and manage it, this red-black tree is not the same tree that manages all the arenas.

3. Hoard

The Hoard allocator architecture is less complicated than jemalloc. It has a global heap for all threads and, inside the global heap, it contains superblocks, which are required by the per-processor heaps. When allocation requests come in, the per-processor heap requests a superblock from the global heap and assigns the object on it. When the current superblock is full, it will request another one from the global heap. Freeing the objects in the superblock until empty will result in the global heap reclaiming the superblock from the per-processor heap and setting it as free block for the next request. Similar to jemalloc, it maintains a memory pool per processor, but the threads in the same processor here share the same heap memory pool. In this case, conflicts between threads in the same processor can happen and cost extra time for the lock and unlock thread operations.[3]

4. Tcmalloc

The tcmalloc memory allocation mainly comes from two places: the central heap and the thread cache. For small object memory requests, the program will first try the thread cache, and when the thread cache is insufficient, the allocator will request a portion of the central heap as thread cache. For large object memory requests (> 32K), the thread must request space from the central cache directly. The cache is organized as a single-linked list, where each element of the array is a single-linked list as well, and each element of the list has the same size.[4]

5. mimalloc

Mimalloc implements some new approaches. The first new thing is free list sharding. It divides the memory into pages, and, for each page, it will have the ability to assign a certain memory size, and only allocations of that size will be assigned to that page. Each page has a free list. For object allocation, it will keep looking for blocks in the same page until this page doesn't have space for object allocation. This can help to reduce the internal fragmentation. Also, for multithread, it has a mechanism called free list multi-sharing. This mechanism asks each thread to maintain a list not only of the page(s) it has, but also a list for the pages of the other threads. When one thread updates its page, malloc or free, that thread will update its list and share to other threads. Other threads will do a batch job to keep the list updated. This mechanism efficiently helps to improve communication between thread memory allocations and avoid the time-consuming lock and unlock operations.[2]

III. METHODOLOGY

In this project, we are using the MOC, the Mass Open Cloud, as our platform to run and test all the allocators and benchmarks. The MOC provides maximum accessibility for us to install and modify the system as we want. This project can also be done on a local PC but the accessibility will be stricter because our project result may vary among different architectures, different size of memory, different OS, etc.
In MOC, we set up a VM with 4 vCPU and 32GB RAM. This VM is running Ubuntu GLIBC 2.310. We access this VM using SSH via private key.

Our benchmarks compile and run in C, so we are using gcc 9.3.0 specifically and for the rest of necessary libraries, we follow the script provided by mimalloc-bench to install them all as the latest version.

The allocators are pulled directly from the original Github repositories. We are using jemalloc 5.2.1, mimalloc v1.7.3. tcmalloc gperftools-2.9.1, hoard 5afe855 and the build-in system malloc for this project. We do not modify any code or any setting for allocators and the OS except to change the LD_PRELOAD for different allocators to change the default allocation policy to certain allocators while running the benchmarks.

For each benchmark, the expected output has elapsed time, resident set size, user time, system time, page faults and page reclaim. For our project, we mainly focus on the total elapsed time, resident set size and page reclaims. These metrics indicate the speed of allocation operation, total usage during allocation and how full the memory used. To have relatively fair results, we run every benchmark five times, at different times on different days, and take the average value of the runs' results.

1. Benchmarks

We used the mimalloc-bench from daanx. This is a suite of benchmarks for memory allocation. It is developed by Microsoft, but the source code for each benchmark comes from its original source. Microsoft only wrote a script to download all the necessary libraries and components, pull the source code from Github repositories, and set up the environment automatically. It should be a fair benchmark suite, and it is easy to build and run. In order to prove it is fair, we randomly check some of the source code after installation, as well as the script to run the benchmark. After our examination, this Github repo provides fair benchmarks with easy setup.The mimalloc-bench also supports the mutilthread parameter, so we don't need to handle that manually.

The benchmarks we chose, for a single-threaded environment, is espresso, barnes, larson, and lean.[7]

- Espresso:
  - Espresso is a programmable logic array analyzer that helps to reduce the complexity of digital logic gate circuits. This benchmark is a cache aware memory allocation application that can help to test the page reclaim performance when used on a single thread
- Barnes:
  - Barnes is a hierarchical n-body particle solver. It simulates the gravitational forces between 163840 particles. This benchmark is a good way to test the elapsed time for allocating continuously for small objects and free them all at the end.
- LeanN:
  - LeanN is a big real-world workload with intensive allocation operation. Using this benchmark can allow us to see the overall performance for each allocator when they are doing intensive operations and how they handle and manage the memory, especially when the system runs out of heap memory and how efficient the memory can be when doing a page reclaim.
- LarsonN:
  - LarsonN will have a similar approach to LeanN, but, while it is running, it will leave some object on the heap and ask another thread to free it. This benchmark can test if the concurrent operation used by the allocator will or will not cause the lock conflict, which may result in extra time.

For multi-threaded environments, we use malloc-test, mstress, and redis for testing. These benchmarks include multi-threaded coordination tests, multi-threaded stress tests, and realistic server multithreaded request tests.[7]

- Malloc-test
  - malloc-test uses N cores, each of which generates N threads, each of which performs 100*10^6 operations to allocate objects of size 1kib. multithreaded operations are particularly important to the allocator in terms of how well each thread is scheduled. The main direction of this benchmark is how to implement allocation quickly without causing a lock
- Mstress
  - Mstress simulates real-life server operations. mstress will use N threads for intensive operations. what each thread allocates will affect the operations of other threads and will be there for a long time. Also, a block in continuous use will be freed, and a block of a certain size will be freed and the object will be reallocated in it. Overall, this benchmark is a very good way to test the time and total memory usage of a large number of operations by multiple threads.
- Redis
  - Redis is simulating a redis server to make more than 1 million requests. These requests will upload 10 new list elements and then request to take the first element. this work will measure the number of requests per second to test the allocator lookup and response time.

IV. HYPOTHESIS

After reading Microsoft's mimalloc paper, we expected that mimalloc would outperform all the other allocators we planned to test. However, that paper also suggested to us that

jemalloc remained serviceable for most purposes. This is especially true, as jemalloc has a long track record, and its behavior is well known.

V. Expreimental Results & Analysis

We began by running the espresso, Barnes, larson, and lean benchmarks on a single thread, or, rather, we attempted to do so. We were able to force this condition on espresso, Barnes, and lean, but larson pre-empted our attempt by opening its own additional threads. Its results are included for completeness but are not interpreted.

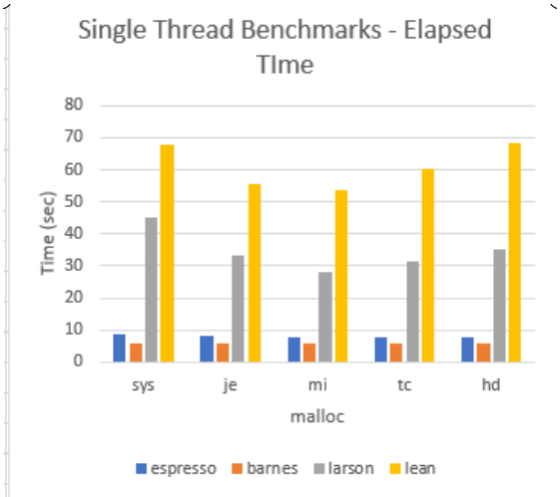


Figure 1. On a single thread, all of the allocators performed similarly, although the system allocator and hoard struggled with the lean benchmark, in particular. (Note that the larson benchmark data is corrupt - it is reported for completeness.)

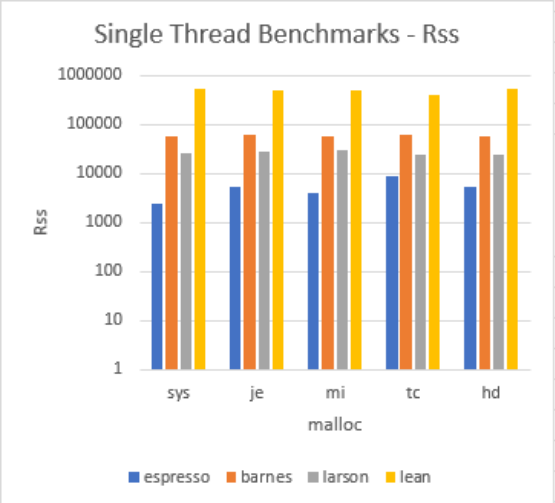


Figure 2. On a single thread, all of the allocators performed similarly. Note the log scale on the y axis. (Note that the larson benchmark data is corrupt - it is reported for completeness.)

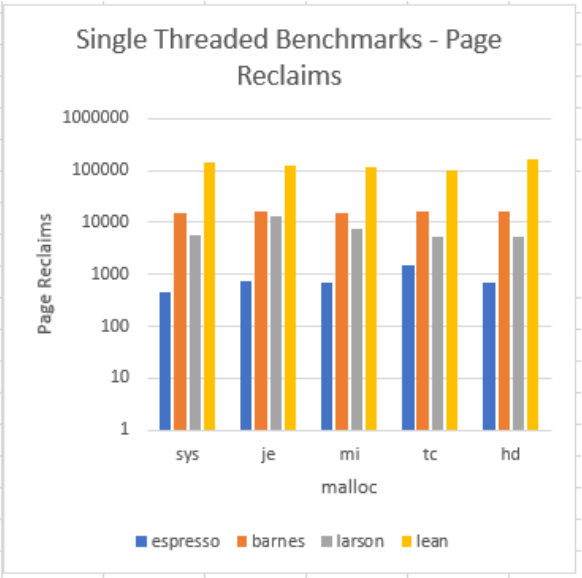


Figure 3. tcmalloc showed some issues with page reclaims on a single thread, which may be due to its thread-local caches, which don't help it on a single thread. Note the log scale on the y axis. (Note that the larson benchmark data is corrupt - it is reported for completeness.)

These preliminary experiments established that the allocators under test were running correctly and with a reasonable degree of stability. Over runs performed over different days and different times of day, there was little variance in our measurements from run to run (for example, the range of values for elapsed time for the mimalloc on barnes was 5.79-5.94 seconds). These experiments also demonstrated that tcmalloc is not terribly effective on a single thread. Since its design revolves around thread-local caches and running on s single thread clearly erases any advantage this design might confer, this is not altogether surprising. In addition, these experiments showed why the system malloc remains the standard. While it was the best only for page reclaims running espresso, it was never the worst of the allocators. The performance of jemalloc, mimalloc, and Hoard were very similar for these experiments. This suggests that a well-implemented allocator (whose advantages are not completely obviated by forcing them onto a single thread) will get the job done, and the best allocator for a particular application will depend on the program's structure and the programmer's priorities.

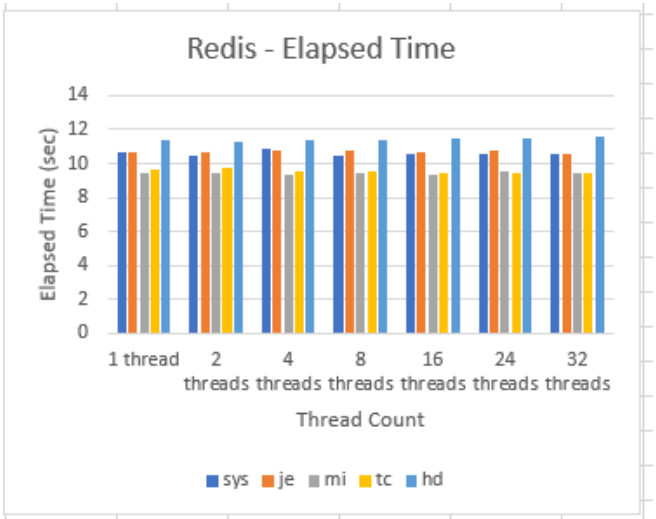


Figure 4. Overall, all the allocators performed well with the Redis benchmark. Runtimes remained close to constant as thread count increased.

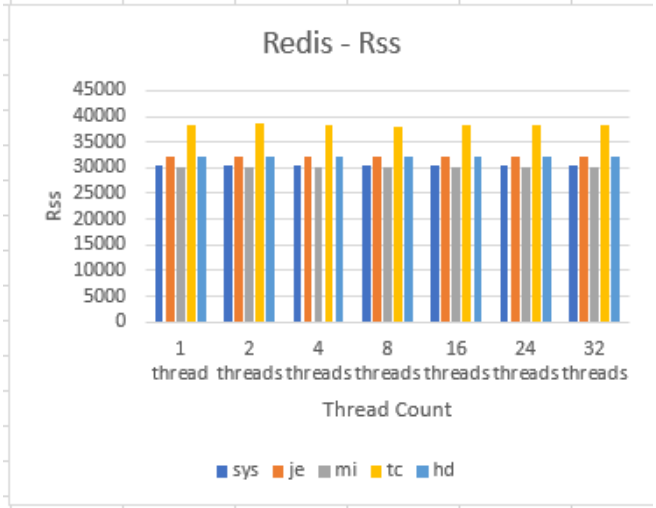


Figure 5. tcmalloc is the worst performer for resident set size (a measure of total memory usage) for Redis. The other allocators perform similarly across thread counts.

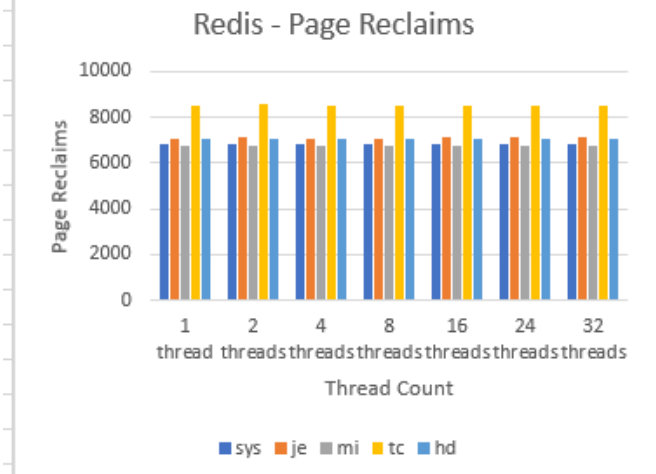


Figure 6. tcmalloc also performs the worst on page reclaims, a measure of how full the memory is, for the Redis benchmark. Again, the others perform similarly.

We then proceeded to test the allocators in a multi-threaded context. In the redis benchmark, which uses a real-world database application, mimalloc did, indeed, give the best performance result across all thread counts. Tcmalloc, so uninspiring when forced onto a single thread, was consistently in second place. The system malloc and jemalloc traded third and fourth place back and forth but were very close throughout. Hoard performed the worst for every thread count. However, Hoard was designed to limit fragmentation and false sharing of cache lines, not necessarily to optimize performance. And to be fair to Hoard, its total memory usage and page reclaims are both solidly middle of the pack. They are also remarkably consistent, not just across runs, but across thread counts, suggesting that its designers' efforts to avoid "blowup" of memory usage were successful. The system allocator and mimalloc had the lowest total memory usage and page reclaims consistently. This suggests that mimalloc's three tier list design works very well under real-world conditions. It also explains why, despite its age, the system allocator (it is based on ptmalloc, first introduced in the 1990s, though updated regularly since) remains standard. Jemalloc equals Hoard's memory usage and page reclaims, suggesting that its use of size categories and assigning new memory allocations according to those categories keeps memory usage and page reclaims reasonable in real-world situations. Tcmalloc's memory usage and page reclaims are not egregious for this real-world application, but they are the highest of the allocators, which may be down to the maintenance of the thread-local caches in addition to a central heap and the movement of data between them.

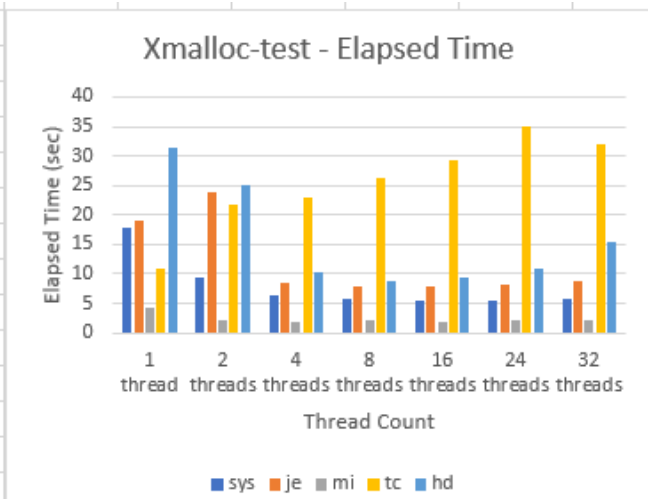


Figure 7. Mimalloc performs the best on the xmalloc-test benchmark. Tcmalloc starts well, but then struggles, while the rest spike before settling into reasonable runtimes.

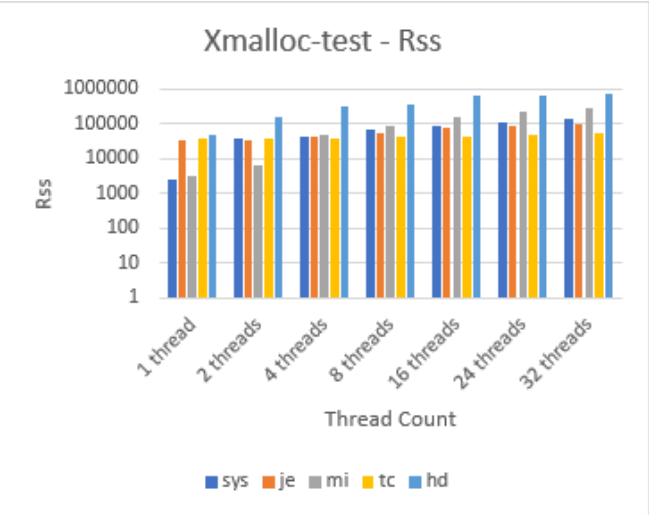


Figure 8. Hoard and mimalloc struggle more than expected with Rss on the xmalloc-test benchmark. Note the log scale on the y axis.

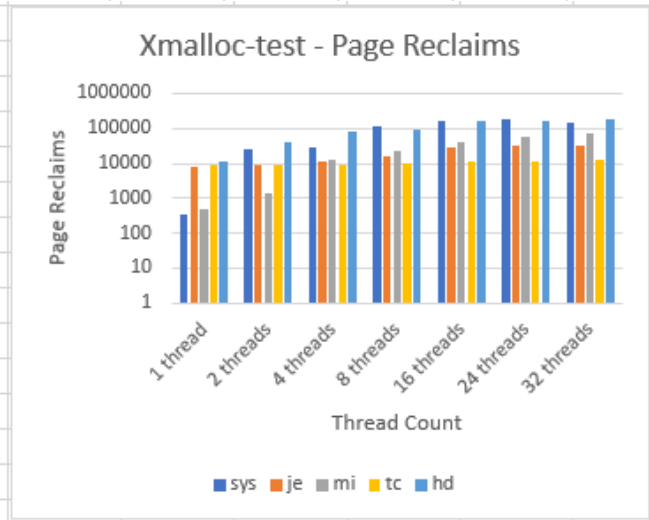


Figure 9. Hoard still struggles more than expected with page reclaims on the xmalloc-test benchmark. Note the log scale on the y axis.

We next tested the allocators with xmalloc-test, a benchmark designed to stress allocators with thread-local caches, as it has 100 allocating threads and 100 deallocating threads, a pattern allocator with thread-local caches find difficult. Given this, it is not surprising that tcmalloc did not perform well overall, although both jemalloc and Hoard performed worse for one and two thread runs. However, jemalloc's runtime dropped significantly at four threads, as did hoards, albeit not quite as dramatically. The system allocator struggled a bit at one thread, but then settled into second place. Moreover, by eight threads, the system allocator's runtime had stabilized at between five and six seconds, again demonstrating why it remains the standard allocator. Mimalloc was the fastest allocator by a factor of two or three across all thread counts, likely because of its lack of locks, a known bottleneck. However, while its memory usage and page reclaim are good at one and two threads, these jump at four threads and proceed to increase rapidly. While it doesn't take last place at any thread count, this is a potentially worrying sign. While jemalloc's memory usage and page reclaims also increase as the number of threads increases, its rate of increase is not nearly as steep. Tcmalloc, despite the fact that this benchmark is designed to stress allocators with thread-local caches, actually has the lowest memory usage and page reclaims of any of the allocators, roughly half the memory usage of jemalloc (second best) at thirty-two threads, and a third of jemalloc's (again, second place) page reclaims at thirty-two threads. The really interesting result is that Hoard, designed to limit memory blowup, has the worst memory usage and page reclaims of any of the allocators. Presumably, the memory usage and page reclaims remain under the limit hoard sets, but in the article that initially described Hoard, their blowup prevention techniques were described in a way that suggested Hoard should use memory in a more judicious manner than is shown here. That said, and in fairness to Hoard, like tcmalloc, it has a thread-local component to its structure, and that may be the reason it struggles here.

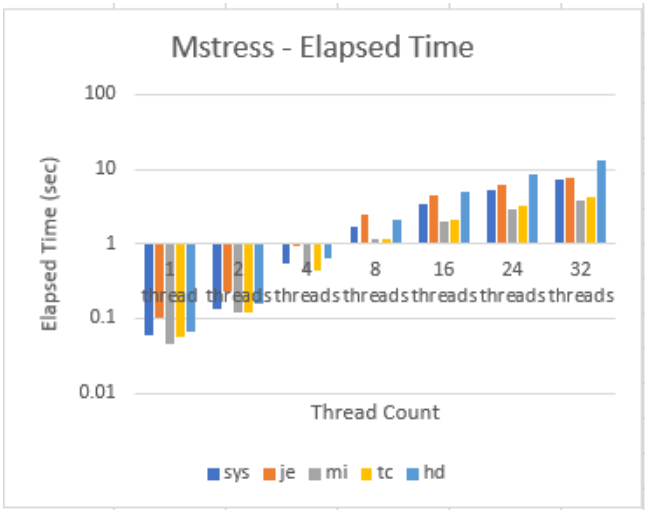


Figure 10. All of the allocators performed fine on the mstress benchmark, though hoard began to spike at 32 threads. Note the log scale on the y axis.

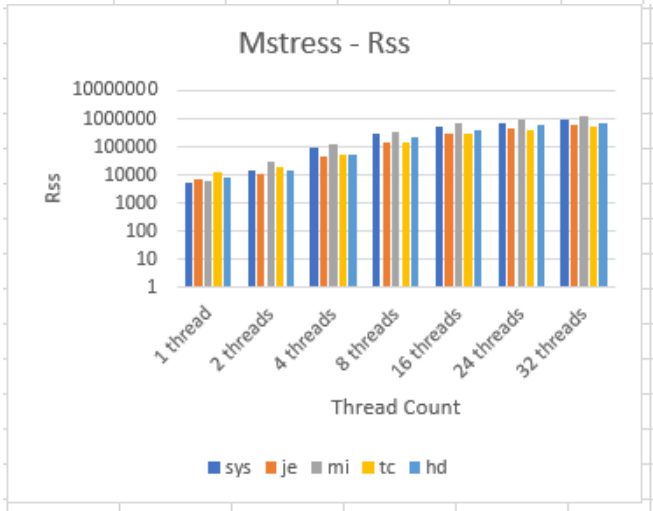


Figure 11. mimalloc struggled with memory usage with the mstress benchmark. Note the log scale on the y axis.

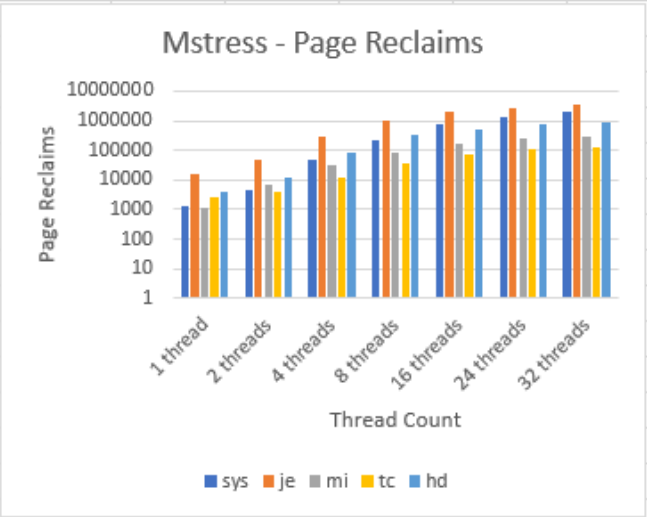


Figure 12. Despite its struggles with memory usage, mimalloc did better with page reclaims with the mstress benchmark. Jemalloc struggled, though. Note the log scale on the y axis.

Lastly, we tested the allocators with the mstress benchmark, which simulates typical server allocation patterns, including various size allocations and allocations that vary in lifetime. All of the allocators performed similarly, for the most part, although Hoard started to suffer at thirty-two threads. The system allocator and jemalloc also saw slowdowns relative to mimalloc and tcmalloc at thirty-two threads, but still performed significantly better than hoard. Overall, mimalloc performed best, but tcmalloc was a close second (even beating mimalloc at eight threads). Hoard, surprisingly, continued to suffer from higher-than-expected memory usage and page reclaims, though it was not the worst at either. Again, it may just be that the limit hoard's designers implemented only caps memory usage and doesn't prevent applications from using memory space unless and until it reaches that cap. As with xmalloc-test, mimalloc's memory usage starts quite low, but then escalated rapidly with the addition of threads, taking last place even at two threads. However, its page reclaims remain quite low relative to the other allocators, taking first or second place for all thread counts. Tcmalloc, the allocator trading first and second place with mimalloc for page reclaims, also has the lowest memory usage for all thread counts, suggesting that a server farm concerned with memory usage should take a look at tcmalloc. Both the system allocator and jemalloc do well with memory usage. Indeed, jemalloc takes second place across all thread counts. However, jemalloc's page reclaims is the worst for all thread counts, starting 5-10x higher than the others', and not improving its relative performance in the category as the number of threads increases. This suggests that a server farm owner might not want to implement jemalloc in its data center.

VI. Conclusion

As with so much in computer engineering, the best choice of allocator depends on what a programmer wants to accomplish and/or the context in which the application will operate. However, our tests show that jemalloc, at fifteen years old, remains a good choice of allocator for general purpose computing in multi-threaded contexts. Mimalloc shows tremendous promise, but its memory usage and page reclaims (not reported in the original paper) may be worrisome, so caution may be warranted until its behavior is better understood. The system allocator, while rarely the best in our tests, was solid and stable in all of the tests, explaining its continued use as the default allocator in libc. Tcmalloc is solid, in specific contexts, and so may be the best choice in those contexts, for example, in server farms. Hoard performed significantly less well than expected overall, but it may be that our choice of benchmarks did not capture its optimal use case.

References

[1] Evans,J.,2006 A Scalable Concurrent malloc(3) Implementation for FreeBSD

[2] LEIJEN, D., ZORN, B. and DE MOURA, L., 2019, Mimalloc: Free List Sharding in Action

[3] Emery D. Berger, Kathryn S. McKinley, Robert D. Blumofe, and Paul R. Wilson. 2000. Hoard: a scalable memory allocator for multithreaded applications. SIGARCH Comput. Archit. News 28, 5 (Dec. 2000), 117–128. DOI:https://doi.org/10.1145/378995.379232

[4] Sanjay Ghemawat. TCMalloc : Thread-Caching Malloc. Retrieved December 10, 2021 from http://pages.cs.wisc.edu/~danb/google-perftools-0.98/tcmalloc.html

[5] Poul-Henning Kamp. 1998. Malloc(3) revisited. In Proceedings of the annual conference on USENIX Annual Technical Conference (ATEC '98). USENIX Association, USA, 36.

[6] En.wikipedia.org. 2021. C dynamic memory allocation - Wikipedia. [online] Available at: <https://en.wikipedia.org/wiki/C_dynamic_memory_allocation> [Accessed 10 December 2021].

[7] Leijen, D., 2021. GitHub - daanx/mimalloc-bench: Suite for benchmarking malloc implementations.. [online] GitHub. Available at: <https://github.com/daanx/mimalloc-bench> [Accessed 10 December 2021].

Division of Labor:

Set up (allocator and benchmark installation on MOC): Niantong

Literature Review (How do allocators work? Find and read relevant articles on specific allocators to use. Find relevant GitHub pages.): Both – shorter articles; Margot – read long historical review of allocator evaluation; Niantong – GitHub pages

Baseline Benchmark experiment (malloc(3) for all benchmarks) & get familiar with how to run the experiment: Both

First runs of benchmarks to ensure functionality: Niantong

Subsequent runs of benchmarks; number crunching and graphing: Margot

Report and Presentation: Both